

U1
-
Introducción al despliegue web y arquitectura cliente-servidor

2° DAW

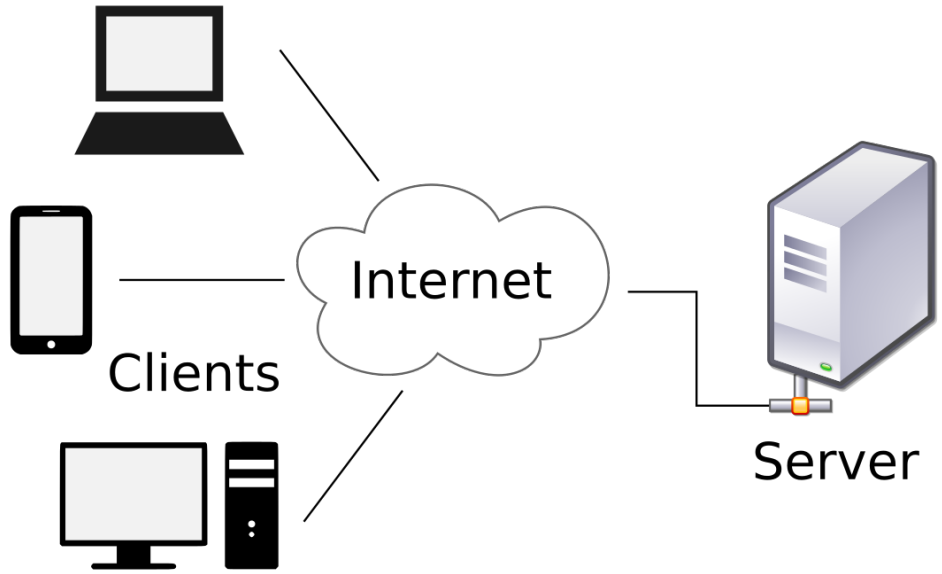
2025/2026

Juanra Collado – jr.colladoesteve@edu.gva.es

RA's que se trabajan en esta unidad

- RA1 - Implanta arquitecturas web analizando y aplicando criterios de funcionalidad
  - Este RA también se trabaja en la U2 y U4.
- RA2 - Implanta aplicaciones web en servidores web, evaluando y aplicando criterios de configuración para su funcionamiento seguro.
  - Este RA también se trabaja en la U2, U3 y U4.

Introducción al despliegue web



Introducción al despliegue web

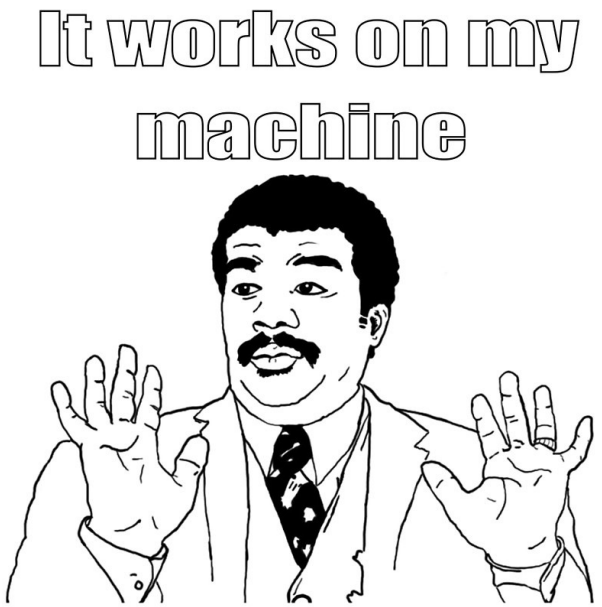
- Desplegar una aplicación web significa hacer que una aplicación o página creada por un desarrollador esté accesible desde internet para otros usuarios.

- No confundamos conceptos:
  - **Programar**: escribir código (HTML, Python, JS, PHP, etc.)
  - **Desplegar**: instalar, configurar y publicar la aplicación en un servidor web para que cualquier pueda acceder a ella a través de un navegador web.

Introducción al despliegue web

- Cuando desarrollamos una aplicación web y la probamos en local, ésta solo es accesible desde el propio PC desde donde estamos trabajando.

- Para que sea accesible por otras personas, es necesario subirla a un servidor web.
  - Y a menudo lo que en local nos funcionaba, falla en el servidor remoto



Componentes básicos de un despliegue web

- Para desplegar una web necesitamos varios componentes básicos.
  - **Software cliente**: Software o dispositivo que realiza las peticiones (suele ser un navegador)
  - **Servidor web:** Software que gestiona las peticiones realizadas por el cliente. El más popular es Apache, aunque últimamente el servidor Nginx está ganando en popularidad por su ligereza y sencillez.
    - En ocasiones podemos tener más servidores (la base de datos puede estar en otro servidor, por ejemplo, o podemos tener varios servidores web para balancear la carga)







Componentes básicos de un despliegue web

- **Aplicación (código):** Ficheros y código que forman la web. Aquí podemos tener código de parte cliente (DWEC) y de parte servidor (DWES).
  - Ambas partes se alojan en el servidor web, tanto la parte cliente como la parte servidor.
  - La principal diferencia es que la parte cliente se ejecuta en el cliente (navegador) una vez se ha descargado desde el servidor y la parte de código de servidor se ejecuta sobre el propio servidor.
- **Red/Internet:** Es necesaria una red para conectar las partes implicadas. Esta red puede ser una red local o Internet.
  - Si es una red local, solo tendrán acceso aquellos usuarios que se conecten desde esta red local.









- Sin al menos estos 4 elementos, no hay despliegue posible.

¿Dónde podemos desplegar?

- Servidor físico propio
  - Más control
  - Más coste
  - Mantenimiento complejo



- VPS (Virtual Private Server)
  - Servidor virtualizado dentro de uno físico compartido (OVH, IONOS, etc)
  - Económico y flexible

¿Dónde podemos desplegar?

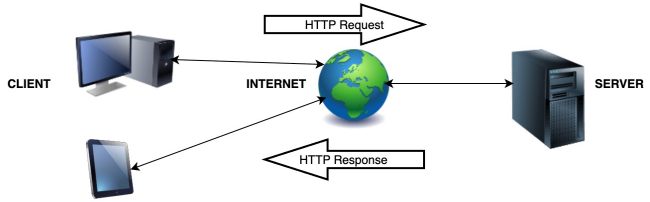
- Contenedores (Docker)
  - Aislamiento a nivel de aplicación
    - Da igual el tipo de servidor que se utilice o la infraestructura de la que se disponga, si el despliegue es en un Docker éste funciona de forma completamente aislada al resto de cosas.
  - El contenedor puede estar incluido dentro de servidores físicos propios, VPS o cloud

- Cloud público (AWS, Azure)
  - Pago por uso
  - Escalabilidad inmediata



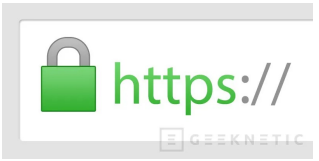
Arquitectura Cliente - Servidor

- Flujo básico:
  - Cliente escribe una URL
    - En la URL indica el recurso que se solicita
  - El navegador genera una petición HTTP
    - También puede ser HTTPS (petición con capa de seguridad añadida)
  - El servidor responde con un documento
    - Puede ser una web HTML, un JSON u otro tipo de documento
  - El navegador interpreta la respuesta y la muestra



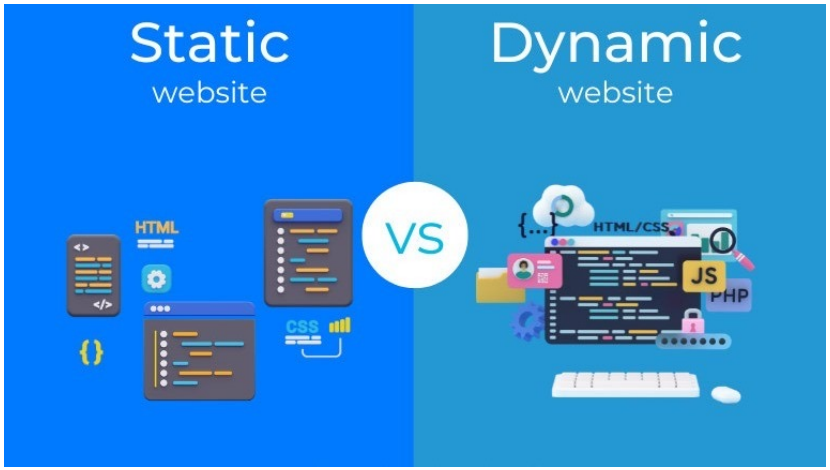
Protocolos: HTTP y HTTPS

- HTTP: HyperText Transfer Protocol
  - Protocolo básico de la web
- HTTPS: HyperText Transfer Protocol Secure
  - Versión segura de HTTP
  - Usa SSL/TLS para cifrar datos
  - Ventajas
    - Seguridad de los datos
    - Confianza de los sitios



Web estática VS Web dinámica

- Una web estática es aquella en la que el contenido es el mismo independientemente de cuando o como accedamos
  - Las que creamos en lenguaje de marcas, son webs estáticas
- Una web dinámica es aquella en la que el contenido puede variar dependiendo de cuando accedamos (tiene acceso a base de datos, API's, etc).
  - Las que crearéis este año en DWES son webs dinámicas



Práctica 1

- Instalar un servidor web Apache en local y desplegar la primera web estática.
  - Crea una MV con VirtualBox y un SO Ubuntu Server sin interfaz gráfica que se llame ApacheMV.
    - La ISO se puede descargar gratuitamente desde la web de Ubuntu.
    - Antes de empezar con la instalación de Apache, se recomienda crear una snapshot de la MV para tener siempre acceso a la máquina sin nada instalado (se utilizará en la siguiente práctica).
  - Instalar Apache
    - sudo apt install apache2
  - Verifica su funcionamiento accediendo a http://localhost
    - Como no tenemos interfaz gráfica, podemos comprobarlo haciendo un *curl http://localhost*
    - Si estamos en modo puente, también podemos acceder desde el navegador del anfitrión a http://IPSERVIDOR
      - http://10.116.65.17
  - Modifica el fichero index.html para que este contenga un título con tu nombre y una breve explicación de la arquitectura cliente-servidor (incluyendo como mínimo una imagen y estilos css en fichero separado).
    - Apache guarda la estructura de directorios web en /var/www/html por lo que buscaremos el index.html allí
  - Verifica su funcionamiento por interfaz gráfica

Práctica 1.1

- Ahora vamos a verificarlo desde un cliente:
  - Asegúrate que la MV está configurada en adaptador puente
  - Copia la IP de la MV
  - Desde la máquina anfitriona (o cualquier otra que esté en la misma red)
    - Conéctate por SSH al servidor
      - *ssh nombredeusuarioservidor@192.168.1.50 //(ip servidor)*
      - Si el servidor da problemas de conexión, nos aseguramos de que habilite conexiones ssh
        - *sudo ufw allow ssh*
        - *sudo ufw enable*
    - Conéctate por FTP al servidor y mueve ficheros (Filezilla)

¿Qué es un servidor web?

- Un **servidor web** es un software que permite entregar páginas web al cliente (navegador) usando protocolos como HTTP/HTTPS.

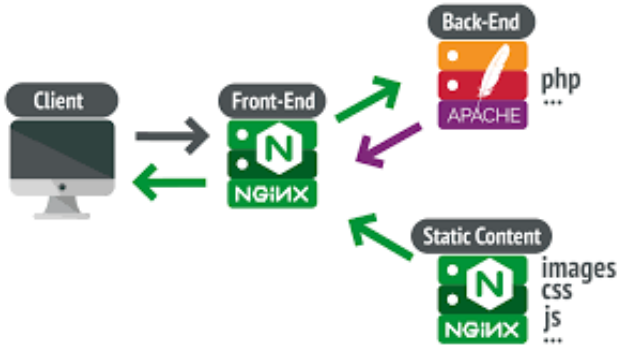
- Funciones principales:
  - Recibir solicitudes HTTP.
  - Procesar la petición (archivos estáticos o aplicaciones dinámicas).
  - Responder con el recurso (HTML, CSS, JS, imágenes, JSON…).
  - Ejemplos: **Apache, Nginx, IIS**.

- Apache → más extendido, flexible.
- Nginx → más ligero y eficiente en altas cargas.

Apache VS Nginx

- Apache:
  - Lanzado en 1995
  - Gran soporte, multiplataforma
  - Configuración basada en archivos .conf y .htaccess
  - Popular para hosting compartido

- Nginx
  - Lanzado en 2004
  - Orientado a rendimiento y concurrencia
  - Ideal como proxy inverso y balanceador de carga
  - Menos consumo de recursos



- Existen infraestructuras de sitios web donde ambos servidores coexisten
  - Por ejemplo, uno sirve el front-end (parte cliente) y otro se encarga de la parte back-end (parte servidor)

Práctica 2

- Instalar un servidor web Nginx en local y desplegar una web estática. Crea una nueva MV con VirtualBox y un SO Ubuntu Server sin interfaz gráfica. Podemos utilizar la misma MV de la práctica 1 desde el snapshot realizando una clonación. Ponle como nombre NginxMV.
  - Busca como instalar Nginx e instálalo.
  - Verifica su funcionamiento accediendo al servidor desde el navegador del anfitrión
  - Modifica el fichero index.html para que este contenga un título con tu nombre y una breve explicación de la arquitectura cliente-servidor (incluyendo como mínimo una imagen y estilos css en fichero separado).
  - Verifica su funcionamiento accediendo al servidor desde el navegador del anfitrión

Archivos de configuración

- **Apache (Debian/Ubuntu):**
  - Global: /etc/apache2/apache2.conf
  - Puertos: /etc/apache2/ports.conf
  - Sitio por defecto: /etc/apache2/sites-available/000-default.conf
  - Habilitar/deshabilitar sitios: a2ensite / a2dissite
  - Servicio: systemctl [status|reload|restart] apache2
- **Apache (CentOS/RHEL):**
  - Global: /etc/httpd/conf/httpd.conf
  - Conf adicionales: /etc/httpd/conf.d/*.conf
  - Servicio: systemctl [status|reload|restart] httpd

Archivos de configuración

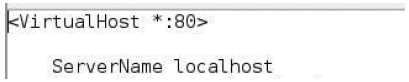
- **Nginx (Debian/Ubuntu):**
  - Global: /etc/nginx/nginx.conf
  - Sitios: /etc/nginx/sites-available/ + enlaces en /etc/nginx/sites-enabled/
  - Servicio: systemctl [status|reload|restart] nginx
- **Nginx (CentOS/RHEL):**
  - Global: /etc/nginx/nginx.conf
  - Conf adicionales: /etc/nginx/conf.d/*.conf
  - Servicio: systemctl [status|reload|restart] nginx

- *Todos los ejemplos se basarán en distribuciones Debian/Ubuntu

Puerto de escucha de los servidores

- Puertos por defecto: 80 (HTTP), 443 (HTTPS)

- Para cambiarlo en Apache
  - *sudo nano /etc/apache2/ports.conf*
  - *Listen 8080 (o el puerto que queramos)*
  - *Es posible que también tengamos que modificar el puerto del VirtualHost correspondiente al sitio web por defecto (000-default.conf)*
    - *Sustituir el puerto en <VirtualHost *:80> -> Sustituir por puerto deseado*



- Para cambiarlo en Nginx (sin vhosts)
  - Sobre el fichero */etc/nginx/sites-available/default*
    - *listen 80 default_server;*
    - *listen [::]:80 default_server;*

Directorios raíz de los servidores

- Directorios por defecto
  - Apache -> /var/www/html
  - Nginx -> /usr/share/nginx/html

- Tanto en Apache como en Nginx, es importante que la carpeta que contiene el sitio tenga los permisos y propietarios adecuados:
  - *Apache*
    - *sudo chown -R www-data:www-data /var/www/miproyecto*
  - *Nginx*
    - *sudo chown -R nginx:nginx/var/www/miproyecto*

Directorios raíz de los servidores

- Para cambiarlo en Apache
  - *sudo nano /etc/apache2/sites-available/000-default.conf*
  - *<VirtualHost *:80>*
    DocumentRoot /var/www/miproyecto
    </VirtualHost>

- Para cambiarlo en Nginx (sin vhosts)
  - Sobre el fichero */etc/nginx/sites-available/default*

Logs y errores típicos

- Cuando el servidor falla, por pantalla muestra muy poca información (o ninguna), es por ello que nos apoyaremos en ficheros de logs que generan los servidores donde encontraremos información de utilidad.
  - Apache: /var/log/apache2/access.log y error.log
  - Nginx: /var/log/nginx/access.log y error.log

```
bob@linux-ub:/var/www/html$ cat /var/log/apache2/error.log
[Sun Mar 27 20:27:00.269176 2016] [mpm_event:notice] [pid 23343:tid 140057225037696
] AH00489: Apache/2.4.7 (Ubuntu) configured -- resuming normal operations
[Sun Mar 27 20:27:00.269264 2016] [core:notice] [pid 23343:tid 140057225037696] AH0
0094: Command line: '/usr/sbin/apache2'
[Mon Mar 28 17:51:40.796855 2016] [mpm_event:notice] [pid 23343:tid 140057225037696
] AH00491: caught SIGTERM, shutting down
[Mon Mar 28 17:51:41.849873 2016] [mpm_event:notice] [pid 52009:tid 140535007307648
] AH00489: Apache/2.4.7 (Ubuntu) configured -- resuming normal operations
[Mon Mar 28 17:51:41.849978 2016] [core:notice] [pid 52009:tid 140535007307648] AH0
0094: Command line: '/usr/sbin/apache2'
[Mon Mar 28 17:53:49.477402 2016] [mpm_event:notice] [pid 52009:tid 140535007307648
] AH00491: caught SIGTERM, shutting down
[Mon Mar 28 17:53:49.530379 2016] [mpm_event:notice] [pid 53542:tid 140182453340032
] AH00489: Apache/2.4.7 (Ubuntu) configured -- resuming normal operations
[Mon Mar 28 17:53:49.530507 2016] [core:notice] [pid 53542:tid 140182453340032] AH0
0094: Command line: '/usr/sbin/apache2'
[Tue Mar 29 20:52:09.189540 2016] [mpm_event:notice] [pid 53542:tid 140182453340032
] AH00491: caught SIGTERM, shutting down
```

Práctica 3

- Modifica el puerto por defecto del sitio Apache y Nginx que tenemos creado.

- Crea una carpeta llamada miproyecto y haz que esta sea la carpeta por defecto que van a buscar tanto Apache como Nginx. Mantén los puertos del ejercicio anterior.

Virtual Hosts (Apache)

- Los virtual host nos permiten alojar varias webs en un mismo servidor.

- Para ello, es necesario realizar cierta configuración porque hasta ahora solo teníamos un punto de acceso:
  - http://localhost:80

- Ahora necesitaremos o diferentes nombres o diferente puerto (o ambos).

Virtual Hosts (Apache)

- Los virtual host nos permiten alojar varias webs en un mismo servidor.

- Para ello, es necesario realizar cierta configuración porque hasta ahora solo teníamos un punto de acceso:
  - http://localhost:80

- Ahora necesitaremos o diferentes nombres o diferente puerto (o ambos).

Virtual Hosts (Apache)

- Para ello es necesario editar el fichero */etc/hosts* donde indicaremos nuestros nombres de dominio (tantos como sitios queramos alojar en nuestro servidor) y la dirección IP donde están estos dominios.

- Estamos simulando un DNS en nuestro propio PC:
  - Las IP's siempre serán la de localhost (127.0.0.1).
  - La dirección URL será la que decidamos (en un entorno real, serían las reales)

```
127.0.0.1  empresa1.daw
127.0.0.1  empresa2.daw
```

Virtual Hosts (Apache)

- Crearemos las carpetas que contendrán nuestros sitios web
  - Es una buena práctica que siempre estén bajo */var/www*
    - sudo mkdir -p /var/www/empresa1 /var/www/empresa2
  - Modificamos los permisos de las carpetas
    - sudo chown -R www-data:www-data /var/www/empresa1 /var/www/empresa2

- Siempre que realicemos acciones de este tipo, hay que asegurarse que se han realizado con éxito.
  - En este caso, la mejor forma es mediante un *ls -l /var/www/*

Virtual Hosts (Apache)

- Crearemos un fichero de configuración por sitio en */etc/apache2/sites-available*
  - *empresa1.conf y empresa2.conf*

- Aquí tenéis un ejemplo de fichero:

```
<VirtualHost *:80>
    ServerName empresa1.daw
    DocumentRoot /var/www/empresa1
    ErrorLog ${APACHE_LOG_DIR}/empresa1_error.log
    CustomLog ${APACHE_LOG_DIR}/empresa1_access.log combined
</VirtualHost>
```

Virtual Hosts (Apache)

- Una vez creadas ambos ficheros y rellenados, es necesario indicarle a Apache que hemos creado 2 sitios. Para ello utilizaremos la instrucción *a2ensite*
  - *sudo a2ensite empresa1.daw.conf*
  - *sudo a2ensite empresa2.daw.conf*

- Y por último, si no hay ningún error, recargaremos Apache para que cargue todos los cambios realizados y podremos acceder a nuestros sitios utilizando las URL configuradas previamente
  - *sudo apachectl configtest*
  - *sudo systemctl reload apache2*

Buenas prácticas

- Un DocumentRoot por sitio.
- Un .conf por sitio y nombre coherente (empresaX.conf).
- Logs separados por sitio para depurar más rápido.
- Comprueba sintaxis antes de recargar:
  - apachectl configtest / nginx -t

Práctica final

- El enunciado de la práctica final de la U1 está en aules y en él:
  - Configurarás servidor nginx multisitio.
  - Indagarás en más parámetros de configuración de los servidores.
  - Incluiremos en una misma máquina dos servidores funcionando en paralelo y sirviendo diferentes páginas.

¿Dudas?

